Хранилище продаж и маркетинга в Greenplum

Илья Партанский. Аналитические базы данных и хранилища

Модель и запуск. Решение проверено в образе andruche/greenplum:7, Greenplum 7.0.0, PostgreSQL 12.12, два primary-сегмента. Внутри контейнера запустить psql от gpadmin. Сначала выполнить sql/00_database.sql в базе postgres, затем sql/01_layers.sql, 02_mart.sql, 03_reports.sql и 04_optimization.sql в sales_marketing. База должна быть новой. Для повторного наполнения существующей витрины достаточно 02_mart.sql: TRUNCATE и INSERT выполняются в одной транзакции. Полный повторный запуск предполагает отдельную чистую базу.

STG хранит пять исходных сущностей: customers, channels, campaigns, orders, order_items. SQL генерирует 120 клиентов, 4 канала, 12 кампаний, 720 заказов и 2880 позиций. Есть четыре региона, три сегмента, статусы completed/cancelled/returned. Случайные цены позиций формируют случайные суммы заказов через сумму quantity x price; заголовки и позиции согласованы. Период заказов и кампаний - 01.08.2026-29.10.2026. Даты регистрации предшествуют заказам.

DDS содержит dim_customers, dim_channels, dim_campaigns, fact_orders и fact_order_items. Бизнес-идентификаторы служат ключами измерений, поскольку источник один, история атрибутов в условии не требуется. Во всех таблицах есть load_dt и load_src. Загрузка - INSERT ... SELECT из STG без потери заказов и позиций. Отменённые и возвращённые заказы сохраняются в DDS для аудита, но исключаются из KPI.

Физическая организация. Факты распределены по order_id одинакового типа bigint: позиции соединяются с заказами на своём сегменте. Небольшие измерения реплицированы, поэтому их соединения не требуют перераспределения большого факта. STG orders, оба факта и CDM разбиты по месяцам. В позиции DDS добавлена order_dt для отсечения партиций вместе с заказами. Для текущего объёма партиции демонстрируют модель роста и управление историей; дополнительные вторичные индексы не нужны. Индексы первичных ключей обеспечивают уникальность. В партиционированных таблицах ключ включает дату, а отсутствие повторов бизнес-идентификаторов дополнительно проверено по данным. Перед новым месяцем потребуется добавить партицию.

CDM распределена по campaign_id. Её гранулярность - кампания и день, канал однозначно определяется кампанией. Первичный ключ (campaign_id, report_dt) исключает повтор группы. Дневной календарь включает дни без завершённых заказов: рекламный расход не исчезает вместе с отсутствием продаж. Получено 1080 строк.

Определения показателей. orders_count - число completed-заказов, customers_count - уникальные покупатели внутри дня и кампании; order_amount и revenue - валовая сумма этих заказов. avg_order_amount - revenue / orders_count, NULL при отсутствии заказов. new_customers - число клиентов, у которых это первый completed-заказ во всей истории. При совпадении дат выбирается минимальный order_id, поэтому клиент атрибутируется ровно одной кампании. Это определение первой покупки, а не регистрации.

Расходы и отчёты. Отдельного источника рекламного биллинга в задании нет. Дневной фактический расход моделируется независимо от продаж: budget / число дней кампании x (0.7 + (campaign_id % 4) x 0.15) x (0.8 + ((campaign_id + номер дня) % 5) x 0.1). Это учебная модель затрат, а не наблюдённый внешний биллинг. Она даёт кампании с расходом ниже и выше бюджета. Расход округляется до копеек на день. campaign_budget - полный бюджет кампании, повторённый на каждой дате; при свёртке применяется MAX, затем бюджеты разных кампаний можно суммировать.

ROI = (revenue - spend) / spend, доля, NULL при нулевом расходе. Отчёты пересчитывают ROI из сумм, не усредняют дневные отношения. Четыре запроса по CDM показывают топ-5 ROI, затраты и бюджет, новых клиентов и стоимость привлечения, показатели каналов. Средний чек канала - суммарная выручка / число заказов. Дневные customers_count не суммируются как число уникальных людей за период; new_customers аддитивен благодаря однократной атрибуции.

Планы и оптимизация. После загрузки выполнен ANALYZE всех STG/DDS и витрины. В 04_optimization.sql одинаковое соединение заказов и позиций сравнивается с временной копией позиций, распределённой RANDOMLY. В исходном плане есть Redistribute Motion 2:2 для позиций. При DISTRIBUTED BY (order_id) этот узел исчезает; остаётся Gather Motion для итогового результата. Условие сентября на обоих фактах читает одну партицию из трёх. В последнем чистом запуске время EXPLAIN ANALYZE составило 6.355 мс для случайного распределения и 2.411 мс для совместного распределения с отсечением партиций. Это один учебный замер, включающий различие в фильтрации позиций; он не изолирует вклад каждой оптимизации и не прогнозирует промышленное ускорение.

Агрегация факта по campaign_id требует Redistribute Motion, поскольку факты распределены по заказам. Агрегация CDM с тем же ключом распределения кампании выполняется локально и затем собирается Gather Motion; время последнего замера 0.503 мс. В сентябре витрина включает также дни без продаж, поэтому её число строк не обязано быть меньше отфильтрованного факта на маленькой выборке. На больших объёмах выигрыш даёт переход от многих заказов к фиксированной сетке день/кампания. В текущих фактах распределение равномерно: 360 заказов на каждом сегменте.

Проверка. Выполнены первоначальный запуск и повторное полное развёртывание в чистой sales_marketing. Все пять SQL-файлов выполнились без ошибок. Собственный автоматизированный тест на Python независимо пересчитал по STG и сверил каждую из 1080 строк CDM: количество заказов и покупателей, выручку, средний чек, новых клиентов, расходы и ROI. Проверены 120 первых покупателей, уникальность идентификаторов, ссылки, суммы позиций и перенос всех строк в DDS со служебными полями. Собственные проверки хранятся отдельно от комплекта сдачи.